**読む問題：次のプログラムの実行結果を入力してください.**
**書く問題：次のプログラムをエディタに書き写して実行し，プログラムと実行結果を入力してください.**

**解答の際, 出力の末尾の改行を入力する必要はありませんが, 途中の改行は正しく入力するようにしてください.**

**問 1〜問 12，問 13〜問 16，問 17〜問 20 の 3 部構成です.**
**各セクションの問題が解き終わったらフォームを送信し, 送信後の画面に記載のリンクから次の問題に解答してください．また，分からない設問があれば解答しなくても構いません.**

## 問１（四則演算）

```
表示する(24%7+4*5)
```

## 問２（変数）

```
x = 10
x = x + 5
表示する(x)
```

## 問３（変数）

```
b = 10
h = 7
a = b * h / 2
表示する(a)
```

## 問４（条件分岐）

```
a = 10
b = 3
もし a > b ならば:
└ a = a - b
a = a * b
表示する(a)
```

## 問５（条件分岐）

```
a = 12
もし a < 12 ならば:
│ 表示する("c")
そうでなければ:
└ 表示する("a")
```

## 問６（条件分岐）

```
s = 30
もし s >= 70 ならば:
│ 表示する("A")
そうでなくもし s <= 30 ならば:
│ 表示する("B")
そうでなければ:
└ 表示する("C")
```

## 問７（繰り返し１）

```
i = 1
i < 5 の間繰り返す:
│ 末尾に空白を入れて表示する(i)
└ i = i + 1
表示する()
```

**問８**（繰り返し１）

```
c = 0
i = 1
i < 256 の間繰り返す:
│ i = i * 2
└ c = c + 1
表示する(c)
```

**問９**（繰り返し２）

```
i を 1 から 3 まで 1 ずつ増やしながら繰り返す:
└ 表示する("こんにちは")
```

**問１０**（繰り返し２）

```
i を 8 から 3 まで 2 ずつ減らしながら繰り返す:
└ 末尾に空白を入れて表示する(i)
表示する()
```

**問１１**（条件分岐と繰り返し）

```
i = 1
i <= 10 の間繰り返す:
│ もし i%2 == 0 ならば:
│ └ 末尾に空白を入れて表示する(i)
└ i = i + 1
表示する()
```

**問１２**（条件分岐と繰り返し）

```
i を 2 から 20 まで 1 ずつ増やしながら繰り返す:
│ cnt = 0
│ j を 2 から i-1 まで 1 ずつ増やしながら繰り返す:
│ │ もし i%j == 0 ならば:
│ │ └ cnt = cnt + 1
│ もし cnt == 0 ならば:
└ └ 末尾に空白を入れて表示する(i)
表示する()
```

**問１３**（リスト）

```
List = [2, 5, 8, 9, 30, 45, 55, 102, 183, 209]
i を 0 から 要素数(List)-1 まで 1 ずつ増やしながら繰り返す:
│ もし i%2 == 1 ならば:
└ └ 末尾に空白を入れて表示する(List[i])
表示する()
```

**問１４**（リスト）

```
List = []
i = 1
i <= 9 の間繰り返す:
│ List.追加(7*i)
└ i = i + 1
表示する(List)
```

**問１５**（関数）

注：整数()関数は引数に指定した数値，文字列を整数に変換して返す関数です．数値の場合は０に近い方に丸められる，つまり，引数が正の場合，切り捨てられます．

```
関数 hasuu(v) を定義する:
│ a = v + 0.5
└ 値 整数(a) を返す
表示する(hasuu(7.4), hasuu(7.5))
```

**問１６**（関数）

```
関数 saiki(n) を定義する:
│ もし n != 0 ならば:
│ │ 値 n*saiki(n-1) を返す
│ そうでなければ:
└ └ 値 1 を返す
表示する(saiki(5))
```

**【　問 13～16 は問 12 までの解答が終わってから取り組んでください　】**

【　問 17～20 は問 16 までの解答が終わってから取り組んでください　】

## 問１７（総合問題）

```
関数 zukei_byouga(Cells) を定義する:
│ i を 0 から 4 まで 1 ずつ増やしながら繰り返す:
│ │ j を 0 から 4 まで 1 ずつ増やしながら繰り返す:
│ │ └ 末尾に空白を入れて表示する(Cells[i][j])
│ └ 表示する()
└ 表示する()
Cells = [[0, 0, 0, 0, 0],
         [0, 0, 0, 0, 0],
         [0, 0, 0, 0, 0],
         [0, 0, 0, 0, 0],
         [0, 0, 0, 0, 0]]
i を 0 から 4 まで 1 ずつ増やしながら繰り返す:
│ j を 0 から 4-i まで 1 ずつ増やしながら繰り返す:
└ └ Cells[i][j] = 1
zukei_byouga(Cells)
```

## 問１８（総合問題）

```
関数 irekae(List, x, y) を定義する:
│ temp = List[x]
│ List[x] = List[y]
└ List[y] = temp
Data = [88, 43, 27, 65]
n = 要素数(Data)
i を 0 から n-2 まで 1 ずつ増やしながら繰り返す:
│ j を n-2 から i まで 1 ずつ減らしながら繰り返す:
│ │ もし Data[j] > Data[j+1] ならば:
└ └ └ irekae(Data, j, j+1)
表示する(Data)
```

## 問１９（総合問題）

```
Data = [8, 15, 24, 55, 64, 77, 85]
s = 64
l = 0
r = 要素数(Data) - 1
f = 0
l <= r の間繰り返す:
│ c = 整数((r + l) / 2)
│ もし Data[c] == s ならば:
│ │ f = 1
│ │ 表示する(c, "番目に見つかりました！")
│ │ ループから抜ける
│ そうでなくもし Data[c] < s ならば:
│ │ l = c + 1
│ そうでなければ:
│ └ r = c - 1
もし f == 0 ならば:
└ 表示する("見つかりませんでした...")
```

**問２０**（総合問題）

```
関数 tugi(Map, posx, posy, dir) を定義する:
│ way = 0
│ もし dir == 0 ならば:
│ │ もし Map[posy][posx+1] != 1 ならば:
│ │ │ posx = posx + 1
│ │ │ もし Map[posy][posx] == 2 ならば:
│ │ │ │ 表示する("→ ゴール！")
│ │ │ │ way = 2
│ │ │ そうでなければ:
│ │ │ │ way = 1
│ │ │ │ Map[posy][posx] = 1
│ │ └ └  末尾に空白を入れて表示する("→")
│ そうでなくもし dir == 1 ならば:
│ │ もし Map[posy+1][posx] != 1 ならば:
│ │ │ posy = posy + 1
│ │ │ もし Map[posy][posx] == 2 ならば:
│ │ │ │ 表示する("↓ ゴール！")
│ │ │ │ way = 2
│ │ │ そうでなければ:
│ │ │ │ way = 1
│ │ │ │ Map[posy][posx] = 1
│ │ └ └  末尾に空白を入れて表示する("↓")
│ そうでなくもし dir == 2 ならば:
│ │ もし Map[posy][posx-1] != 1 ならば:
│ │ │ posx = posx - 1
│ │ │ もし Map[posy][posx] == 2 ならば:
│ │ │ │ 表示する("← ゴール！")
│ │ │ │ way = 2
│ │ │ そうでなければ:
│ │ │ │ way = 1
│ │ │ │ Map[posy][posx] = 1
│ │ └ └ 末尾に空白を入れて表示する("←")
```

**【　プログラムは右に続きます　】**

```
│ そうでなくもし dir == 3 ならば:
│ │ もし Map[posy-1][posx] != 1 ならば:
│ │ │ posy = posy - 1
│ │ │ もし Map[posy][posx] == 2 ならば:
│ │ │ │ 表示する("↑ ゴール！")
│ │ │ │ way = 2
│ │ │ そうでなければ:
│ │ │ │ way = 1
│ │ │ │ Map[posy][posx] = 1
│ └ └ └ 末尾に空白を入れて表示する("↑")
└ 値 Map, posx, posy, way を返す
Map = [[1, 1, 1, 1, 1, 1, 1, 1],
      [1, 1, 0, 0, 0, 0, 1, 1],
      [1, 1, 0, 0, 0, 0, 1, 1],
      [1, 0, 0, 1, 1, 2, 0, 1],
      [1, 0, 0, 1, 1, 0, 0, 1],
      [1, 0, 0, 0, 0, 0, 0, 1],
      [1, 1, 1, 1, 1, 1, 1, 1]]
posx = 3
posy = 2
dir = 0
way = 0
way != 2 の間繰り返す:
│ Map, posx, posy, way = tugi(Map, posx,
posy, dir)
│ もし way == 0 ならば:
└ └ dir = (dir + 3) % 4
```

**【　問題は以上です　】**